

Tic Tac Terminal

Final Report

Patrick Ramsay

Catholic Institute of Technology

August 16, 2025

Table of Contents

Abstract.....	3
Background.....	3
Text-Based Game Description.....	3
Main Code Description.....	4
Utility Functions.....	6
Screen Functions.....	7
Game Logic.....	9
Main Loop.....	10
Ultimate Tic Tac Toe Description.....	11
Conclusion.....	11

Abstract

Our group began with a pre-written code that provided a text-based game of Tic-Tac-Toe. Our goal was to create a new user interface for the game using the Pygame library, which allowed users to play by simply clicking on a Tic-Tac-Toe board. We also wanted to have three game modes available that the user could choose by clicking one of the buttons. Later on, as a side project, we made a game of Ultimate Tic-Tac-Toe, which is similar to the regular game except that it has one large grid and nine smaller subgrids to win.

Background

My team was given a pre-written code for tic-tac-toe that ran completely inside the terminal with no GUI, accepting only text as input and outputting text as well. We used the PyGame library for our UI and basic game functions. We also utilized the sys library to refine certain aspects of our UI. Our programming language was Python, and we shared our code with Google Colab.

Text-Based Game Description

The text-based code begins with the class Board, which contains five functions. The first function initializes the class. The second function displays the board, which utilizes a for loop to produce a numbered grid with the characters '-' for rows, '|' for columns, and '+' for corners.

```

      1   2   3
1   |   |
  ---+---+---
2   |   |
  ---+---+---
3   |   |
```

The third function marks the board where the user tells it to based on the coordinates input as long as the space is empty.

```
def mark(self, row: int, col: int, symbol: str) -> bool:
    # Place symbol if cell is empty
    if self.grid[row][col] == " ":
        self.grid[row][col] = symbol
        return True
    return False
```

The fourth function checks to see who the winner is by checking if there is a full column, row, or diagonal. If none of these conditions are satisfied, it returns None.

```
def check_winner(self) -> Optional[str]:
    lines = []
    # Rows, columns, diagonals
    lines.extend(self.grid)
    lines.extend([[self.grid[r][c] for r in range(3)] for c in range(3)])
    lines.append([self.grid[i][i] for i in range(3)])
    lines.append([self.grid[i][2-i] for i in range(3)])
    for line in lines:
        if line[0] != " " and all(cell == line[0] for cell in line):
            return line[0]
    return None
```

The final function in the class checks to see if the grid is full.

```
def is_full(self) -> bool:
    return all(cell != " " for row in self.grid for cell in row)
```

Skipping past the game modes and menu display, the text-based game has the main function with a while True loop that receives the user's inputs, marks the board, and checks if there's a winner based on the previously mentioned functions.

```
def main():
    board = Board()
    opponent_type, ai_level = select_mode()
    current = "X"
    board.display()

    while True:
        if opponent_type == "C" and current == "O":
            # AI turn
            if ai_level == "easy":
                r, c = random_move(board)
            else:
                r, c = minimax_move(board, "O")
            print(f"Computer places O at {r+1}, {c+1}")
        else:
            r, c = get_move_input(current)

        if not board.mark(r, c, current):
            print("Cell occupied, try again.")
            continue

        board.display()
        winner = board.check_winner()
        if winner:
            print(f"Game Over! {winner} wins!")
            break
        if board.is_full():
            print("Game Over! It's a draw.")
            break

        current = "O" if current == "X" else "X"

    if __name__ == "__main__":
        main()
```

Description of Our Code

We first initialized everything in our code and then went on to build the basic graphics for our game. With PyGame's draw.line functions, we created a 3*3 grid to work with.

```
screen.fill("black")
pygame.draw.line (screen, "white", (0,250), (750,250), width = 10)
pygame.draw.line (screen, "white", (0,500), (750,500), width = 10)
pygame.draw.line (screen, "white", (250,0), (250,750), width = 10)
pygame.draw.line (screen, "white", (500,0), (500,750), width = 10)
```

Our next focus was to make our code recognize which grid was clicked on by the user, so we used the gridpos function in the PyGame library to assign areas in the grid we made to numbers 1-9.

```
c1 = [(0,500),(249,750)]
b1 = [(0,250),(249,500)]
a1 = [(0,0),(249,250)]
c2 = [(250,500),(499,750)]
b2 = [(250,250),(499,500)]
a2 = [(250,0),(499,250)]
c3 = [(500,500),(750,750)]
b3 = [(500,250),(750,500)]
a3 = [(500,0),(750,250)]

if inside(p, a1): gridpos = 0
elif inside(p, a2): gridpos = 1
elif inside(p, a3): gridpos = 2
elif inside(p, b1): gridpos = 3
elif inside(p, b2): gridpos = 4
elif inside(p, b3): gridpos = 5
elif inside(p, c1): gridpos = 6
elif inside(p, c2): gridpos = 7
elif inside(p, c3): gridpos = 8
```

Next, our code has a button class for our menu with three functions. The first function initializes the class, the second function draws the button graphic, and the third function checks to see if it's clicked.

```
class Button:
    def __init__(self, text, rect, font, color, bg):
        self.text_surf = font.render(text, True, color)
        self.rect = pygame.Rect(rect)
        self.bg = bg

    def draw(self, surface):
        pygame.draw.rect(surface, self.bg, self.rect, border_radius=5)
        text_rect = self.text_surf.get_rect(center=self.rect.center)
        surface.blit(self.text_surf, text_rect)

    def is_clicked(self, event):
        return (
            event.type == pygame.MOUSEBUTTONDOWN
            and self.rect.collidepoint(event.pos)
        )
```

Utility Functions

After the button function in our code, there are the utility functions.

The first utility function checks the position of where the user clicked.

```
def inside(p, box):  
    (x1, y1), (x2, y2) = box  
    return x1 <= p[0] <= x2 and y1 <= p[1] <= y2
```

The second function checks to see if anyone won. It starts with a list that has every combination of grid positions that result in a win, and then it checks to see if O or X has one of those combinations.

```
def check_win(activeX, activeO):  
    x_pos=[pos_map.index(k) for k in activeX]  
    o_pos=[pos_map.index(k) for k in activeO]  
    possible_wins= [  
        [0,1,2], [3,4,5], [6,7,8], [0,3,6], [1,4,7], [2,5,8], [0,4,8], [2,4,6]  
    ]  
    for h in possible_wins:  
        if all(i in o_pos for i in h):  
            return 'O'  
        elif all(i in x_pos for i in h):  
            return 'X'  
    return None
```

The third function checks to see if there's a draw by returning True if there are nine moves made on the board.

```
def check_draw(activeX, activeO):  
    x_pos=[pos_map.index(k) for k in activeX]  
    o_pos=[pos_map.index(k) for k in activeO]  
    draw_pos=x_pos+o_pos  
    if len(draw_pos)==9:  
        return 9  
    else:  
        return None
```

Screen Functions

The `x_screen`, `o_screen`, and `draw_screen` are the functions that display whether x or o wins, or if there's a draw. Each function looks exactly like the one below, except with a different message.

```
def x_screen():
    buttons = [
        Button("Return to Main Menu", (200, 350, 350, 70), FONT_SMALL, (0,0,0), (192,192,192)),
    ]
    while True:
        for event in pygame.event.get():
            if event.type == pygame.QUIT:
                return "quit"
            for btn in buttons:
                if btn.is_clicked(event):
                    return "menu"
        screen.fill((0,0,255))
        screen.blit(FONT_LARGEST.render("X Won!", True, (0,0,0)), (240, 50))
        screen.blit(FONT_MEDIUM.render("Would you like to play again?", True, (0,0,0)), (80, 200))
        for btn in buttons:
            btn.draw(screen)
        pygame.display.flip()
        clock.tick(60)
```

The `menu_screen` function gives the user three buttons to click. Depending on what's clicked, the function returns either "play", "play_ai_easy", or "play_ai_hard"

```
def menu_screen():
    buttons = [
        Button("Player vs Player", (200, 350, 350, 70), FONT_SMALL, (0,0,0), (192,192,192)),
        Button("Player vs CPU (Easy)", (200, 450, 350, 70), FONT_SMALL, (0,0,0), (192,192,192)),
        Button("Player vs CPU (Hard)", (200, 550, 350, 70), FONT_SMALL, (0,0,0), (192,192,192)),
    ]
    while True:
        for event in pygame.event.get():
            if event.type == pygame.QUIT:
                return "quit"
            for i, btn in enumerate(buttons):
                if btn.is_clicked(event):
                    if i == 0:
                        return "play"
                    elif i == 1:
                        return "play_ai_easy"
                    elif i == 2:
                        return "play_ai_hard"
        screen.fill((0,0,255))
        screen.blit(FONT_LARGE.render("Welcome to Tic-Tac-Toe!", True, (0,0,0)), (20, 100))
        screen.blit(FONT_MEDIUM.render("Choose a player mode:", True, (0,0,0)), (130, 200))
        for btn in buttons:
            btn.draw(screen)
        pygame.display.flip()
        clock.tick(60)
```

Game Logic

We used the variables *activeX* and *activeO*, which are lists that hold the positions for X and O.

The variable *turn* tracks whose turn it is

The cell boundary variables are there for click detection

```
activeX = []
activeO = []
turn = 0
```

The function *get_empty_indices* returns indices of empty cells.

```
def get_empty_indices(activeX, activeO):
    return [i for i in range(9) if pos_map[i] not in activeX and pos_map[i] not in activeO]
```

The function *random_ai_move* picks a random empty cell.

```
def random_ai_move(activeX, activeO):
    empties = get_empty_indices(activeX, activeO)
    return random.choice(empties) if empties else None
```

The function *minimax_ai_move* uses the minimax algorithm for an optimal AI move. Minimax Recursively simulates all possible moves. It assigns scores: +1 for X win, -1 for O win, 0 for a draw. Chooses a move that maximizes (for X) or minimizes (for O) the score.

```
def minimax_ai_move(activeX, activeO, player):
    def minimax(x_moves, o_moves, turn):
        winner = check_win(x_moves, o_moves)
        if winner == "X":
            return 1
        elif winner == "O":
            return -1
        elif len(x_moves) + len(o_moves) == 9:
            return 0
        empties = get_empty_indices(x_moves, o_moves)
```

```
        if turn == "X":
            best = -float("inf")
            for idx in empties:
                score = minimax(x_moves + [pos_map[idx]], o_moves, "O")
                best = max(best, score)
            return best
        else:
            best = float("inf")
            for idx in empties:
                score = minimax(x_moves, o_moves + [pos_map[idx]], "X")
                best = min(best, score)
            return best
```

The following loop is present in whichever game mode the user chooses. It does the following.

- Determines which cell was clicked.
- Draws the grid and pieces.
- Handles player(s) and/or AI moves:
- Checks for a win/draw after each move.
- Returns "won_x", "won_o", or "draw" when the game ends.

```
running = True
while running:
    for event in pygame.event.get():
        if event.type == pygame.QUIT:
            return "quit"
        elif event.type == pygame.MOUSEBUTTONDOWN:
            p = event.pos
            if inside(p, a1): gridpos = 0
            elif inside(p, a2): gridpos = 1
            elif inside(p, a3): gridpos = 2
            elif inside(p, b1): gridpos = 3
            elif inside(p, b2): gridpos = 4
            elif inside(p, b3): gridpos = 5
            elif inside(p, c1): gridpos = 6
            elif inside(p, c2): gridpos = 7
            elif inside(p, c3): gridpos = 8

    screen.fill("black")
    pygame.draw.line (screen, "white", (0,250), (750,250), width = 10)
    pygame.draw.line (screen, "white", (0,500), (750,500), width = 10)
    pygame.draw.line (screen, "white", (250,0), (250,750), width = 10)
    pygame.draw.line (screen, "white", (500,0), (500,750), width = 10)

    for x in activeX:
        impx = pygame.image.load("xttt.png")
        impx = pygame.transform.scale(impx,(230,230))
        screen.blit(impx, x)
    for x in activeO:
        impo = pygame.image.load("ottt.png")
        impo = pygame.transform.scale(impo,(230,230))
        screen.blit(impo, x)

    if gridpos is not None:
        if pos_map[gridpos] in activeX or pos_map[gridpos] in activeO:
            gridpos = None
        elif turn % 2 == 0:
            turn += 1
            impx = pygame.image.load("xttt.png")
            impx = pygame.transform.scale(impx,(230,230))
            screen.blit(impx, pos_map[gridpos])
            activeX.append(pos_map[gridpos])
        else:
            turn += 1
            impo = pygame.image.load("ottt.png")
            impo = pygame.transform.scale(impo,(230,230))
            screen.blit(impo, pos_map[gridpos])
            activeO.append(pos_map[gridpos])
        gridpos = None

    winner = check_win(activeX, activeO)
    if winner is not None:
        if winner == 'X':
            return "won_x"
        elif winner == 'O':
            return "won_o"
    if check_draw(activeX, activeO) == 9:
        return "draw"
    pygame.display.flip()
    clock.tick(60)
```

Main Loop

Our Main Loop controls the overall game state.

Starts at "menu".

```
state = "menu"
```

Switches between the menu, game, AI game, and end screens based on user actions and game results.

```
while state != "quit":
    if state == "menu":
        next_mode = menu_screen()
        if next_mode in ["play_ai_easy", "play_ai_hard"]:
            ai_mode = next_mode
            state = next_mode
        elif state == "play":
            state = game_screen()
        elif state in ["play_ai_easy", "play_ai_hard"]:
            state = ai_game_screen(mode=state)
        elif state == "won_o":
            state = o_screen()
        elif state == "won_x":
            state = x_screen()
        elif state == "draw":
            state = draw_screen()
        else:
            state = menu_screen()
```

Quits Pygame when done.

```
pygame.quit()
```

Ultimate Tic Tac Toe Description.

The Ultimate Tic-Tac-Toe code is similar to the original Tic Tac Toe code, but two important functions make it possible to have nine different games inside of one.

The first is the `Check_sub_board_win`, which is used to check the smaller boards.

```
def check_sub_board_win(activeX, active0):
    # activeX and active0 are now lists of local positions (0-8) within a sub-board
    possible_wins = [
        [0,1,2], [3,4,5], [6,7,8], [0,3,6], [1,4,7], [2,5,8], [0,4,8], [2,4,6]
    ]

    for h in possible_wins:
        if all(i in active0 for i in h):
            return 'O'
        elif all(i in activeX for i in h):
            return 'X'
    return None
```

The second is `check_ultimate_win`, which checks to see if there's a win on the larger grid.

```
def check_ultimate_win(won_boards_X, won_boards_0):
    possible_wins = [
        [0,1,2], [3,4,5], [6,7,8], [0,3,6], [1,4,7], [2,5,8], [0,4,8], [2,4,6]
    ]
    for h in possible_wins:
        if all(i in won_boards_0 for i in h):
            return 'O'
        elif all(i in won_boards_X for i in h):
            return 'X'
    return None
```

Conclusion

In our project, we successfully turned a text-based game of Tic-Tac-Toe into an engaging graphical interface with the Pygame library. We implemented three different game modes in order to cater to various preferences or skill levels. We also made a game of Ultimate Tic Tac Toe as a side project in case the user wanted to play something more advanced. This project also helped my team gain experience in code development and collaboration, which required us to adopt a modular coding style to ensure that everyone could make significant contributions to the game.